Artificial Intelligence & Machine Learning

Complete Documentation for Developers

Student Developer Roadmap

May 28, 2026

Contents

1 What is AI/ML?

1.1 Key Concepts

- **Artificial Intelligence (AI):** Machines simulating human intelligence
- **Machine Learning (ML):** Algorithms that learn from data
- **Deep Learning (DL):** Neural networks with many layers
- **Generative AI:** Creates new content (text, images, code, music)
- **LLM (Large Language Model):** AI for text understanding/generation (GPT, Llama, Claude)

1.2 Types of Machine Learning

Type	Description
Supervised Learning	Learn from labeled examples (classification, regression)
Unsupervised Learning	Find patterns in unlabeled data (clustering)
Reinforcement Learning	Learn through trial and error (game playing, robotics)
Self-Supervised Learning	Learn from unlabeled data using pretext tasks (LLMs)

2 AI Development Methods

Method	Description
API-First AI	Use pre-trained models via APIs (OpenAI, Anthropic, Google Gemini)
Fine-Tuning	Adapt pre-trained models to specific tasks with custom data
RAG (Retrieval-Augmented Generation)	Combine LLMs with external knowledge bases
Transfer Learning	Use pre-trained model as starting point
Prompt Engineering	Design inputs to get desired outputs from LLMs
Agent-Based AI	Multiple AI agents collaborating to solve tasks
Edge AI	Run models on devices (mobile, IoT)

3 Essential Tools for AI Development

3.1 Programming Language

- **Python:** Primary language for AI/ML (learn this first!)

Python + TensorFlow + OpenAI API

- **Jupyter Notebook:** Interactive development environment
- **Google Colab:** Free GPU/TPU in browser

3.2 Machine Learning Libraries

Library	Purpose
NumPy	Numerical computing, arrays
Pandas	Data manipulation and analysis
Matplotlib / Seaborn	Data visualization
Scikit-learn	Classical ML algorithms
TensorFlow	Deep learning (Google)
PyTorch	Deep learning (Meta, more research-focused)
Keras	High-level API for TensorFlow
Hugging Face Transformers	Pre-trained models (BERT, GPT, Llama)
LangChain	Build LLM applications
LlamaIndex	RAG framework for LLMs
OpenAI API	GPT-4, GPT-3.5, DALL-E, Whisper
Anthropic API	Claude models
Google Gemini API	Google's LLM

3.3 Development Environment

- **VS Code:** With Python extension and Jupyter support
- **Anaconda:** Python distribution with data science packages
- **Docker:** Containerized AI apps
- **Weights & Biases:** Experiment tracking
- **MLflow:** ML lifecycle management

3.4 GPUs & Cloud Platforms

- **Google Colab:** Free GPU/TPU (best for learning)
- **Kaggle:** Free GPU, datasets, competitions
- **Hugging Face Spaces:** Host and demo models
- **AWS SageMaker:** Enterprise ML platform
- **Replicate:** Run models via API
- **Modal:** Serverless GPU compute

4 Learning Resources

4.1 Free Resources

Resource	Focus
fast.ai	Practical deep learning (free course)
Hugging Face Course	NLP and transformers
Google Colab Notebooks	Interactive tutorials
Kaggle Learn	Micro-courses on ML
OpenAI Cookbook	Examples for GPT API
Andrej Karpathy's YouTube	Neural networks from scratch
3Blue1Brown	Math behind neural networks

4.2 Paid Courses

- **DeepLearning.AI:** Andrew Ng's courses (Coursera)
- **Fast.ai:** Free but donation-supported
- **DataCamp:** Interactive data science courses
- **DeepMind x UCL:** Reinforcement learning lectures

4.3 Must-Read Books

- "Hands-On Machine Learning with Scikit-Learn & TensorFlow" by Aurélien Géron
- "Deep Learning" by Ian Goodfellow (free online)
- "Pattern Recognition and Machine Learning" by Christopher Bishop

5 Project Sequence for AI/ML

5.1 Level 1 – Python Fundamentals for AI

1. Data analysis with Pandas (clean, filter, aggregate)
2. Data visualization with Matplotlib/Seaborn
3. NumPy array operations
4. Build a simple calculator with Python

5.2 Level 2 – Classical Machine Learning

1. **Linear Regression:** Predict house prices
2. **Logistic Regression:** Email spam classifier
3. **Decision Trees:** Iris flower classification
4. **K-Means Clustering:** Customer segmentation

5.3 Level 3 – Deep Learning

1. **Neural Network from Scratch:** MNIST digit recognition
2. **Image Classifier:** Cats vs dogs with CNN
3. **Text Classifier:** Sentiment analysis with RNN/LSTM
4. **Transfer Learning:** Use ResNet for custom image classification

5.4 Level 4 – LLMs & Generative AI

1. **Chatbot with OpenAI API:** Basic conversational AI
2. **Document Q&A System:** RAG with your own PDFs
3. **Code Generator:** Prompt engineering for code generation
4. **Image Generator:** DALL-E or Stable Diffusion integration

5.5 Level 5 – Production AI Applications

1. **Fine-Tuned Model:** Custom classifier with Hugging Face
2. **AI API Service:** Deploy model as REST API with FastAPI
3. **AI Web App:** Full-stack app with React + Node.js + OpenAI
4. **AI Mobile App:** React Native app with on-device or cloud AI

6 Concentration: Python for AI (Fundamentals)

Listing 1: Essential Python for AI

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# NumPy: Fast array operations
arr = np.array([1, 2, 3, 4, 5])
print(arr.mean(), arr.std(), arr.sum())
```

Python + TensorFlow + OpenAI API

```

# Matrix operations
matrix = np.random.rand(3, 3)
inverse = np.linalg.inv(matrix)

# Pandas: Data manipulation
df = pd.read_csv('data.csv')
print(df.head())
print(df.info())
print(df.describe())

# Filtering
filtered = df[df['column'] > 10]

# Group by and aggregate
grouped = df.groupby('category')['value'].mean()

# Matplotlib: Visualization
plt.figure(figsize=(10, 6))
plt.plot(df['date'], df['sales'])
plt.title('Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.show()

# Seaborn: Statistical visualizations
sns.heatmap(df.corr(), annot=True)
sns.pairplot(df, hue='target')
```

7 Concentration: Scikit-learn (Classical ML)

Listing 2: Complete ML Pipeline with Scikit-learn

```
from sklearn.model_selection import train_test_split, cross_val_score,
    GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
    accuracy_score
import pandas as pd

# Load data
df = pd.read_csv('customer_data.csv')

# Separate features and target
X = df.drop('churn', axis=1)
y = df['churn']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```

# Define preprocessing for numerical and categorical columns
numerical_cols = ['age', 'income', 'usage_months']
categorical_cols = ['plan_type', 'region']

numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ]
)

# Create pipeline
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42))
])

# Hyperparameter tuning
param_grid = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [10, 20, None],
    'classifier__min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1
)

# Train
grid_search.fit(X_train, y_train)

# Best model
best_model = grid_search.best_estimator_
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")

# Evaluate on test set
y_pred = best_model.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

8 Concentration: TensorFlow & Keras (Deep Learning)

Listing 3: Neural Network with TensorFlow/Keras

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Preprocess
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = x_train.reshape(-1, 28*28)
x_test = x_test.reshape(-1, 28*28)

# Build model
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    x_train, y_train,
    batch_size=32,
    epochs=10,
    validation_split=0.2,
    callbacks=[
        keras.callbacks.EarlyStopping(patience=3, restore_best_weights=
            True),
        keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2)
    ]
)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")

# Save model
model.save('mnist_classifier.h5')

# Load and predict
loaded_model = keras.models.load_model('mnist_classifier.h5')
predictions = loaded_model.predict(x_test[:10])
predicted_classes = np.argmax(predictions, axis=1)
```

Python + TensorFlow + OpenAI API

Listing 4: CNN for Image Classification

```
# CNN Model for CIFAR-10
model = keras.Sequential([
    # First convolutional block
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    # Second convolutional block
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # Third convolutional block
    layers.Conv2D(64, (3,3), activation='relu'),

    # Flatten and dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Data augmentation for better generalization
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])

# Model with augmentation
inputs = keras.Input(shape=(32,32,3))
x = data_augmentation(inputs)
x = layers.Conv2D(32, (3,3), activation='relu')(x)
# ... rest of model
```

9 Concentration: Hugging Face & LLMs

Listing 5: Using Pre-trained Transformers

```
from transformers import pipeline, AutoTokenizer,
    AutoModelForSequenceClassification
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Sentiment analysis
classifier = pipeline("sentiment-analysis", model="distilbert-base-
    uncased-finetuned-sst-2-english")
result = classifier("I␣love␣this␣product!␣It's␣amazing.")
print(result)

# Text generation (GPT-2)
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
```

Python + TensorFlow + OpenAI API

```
model = GPT2LMHeadModel.from_pretrained("gpt2")

input_text = "Once upon a time"
inputs = tokenizer.encode(input_text, return_tensors="pt")
outputs = model.generate(
    inputs,
    max_length=100,
    num_return_sequences=1,
    temperature=0.7,
    do_sample=True,
    pad_token_id=tokenizer.eos_token_id
)
generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(generated_text)

# Question answering
qa_pipeline = pipeline("question-answering", model="distilbert-base-
    cased-distilled-squad")
context = "The Eiffel Tower is in Paris, France. It was built in 1889."
question = "Where is the Eiffel Tower?"
answer = qa_pipeline(question=question, context=context)
print(answer)

# Named Entity Recognition
ner_pipeline = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-
    conll03-english")
text = "Apple Inc. is headquartered in Cupertino, California."
entities = ner_pipeline(text)
print(entities)
```

Listing 6: Fine-Tuning a Transformer Model

```
from transformers import AutoTokenizer,
    AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import Dataset
import pandas as pd
import numpy as np

# Load custom dataset
df = pd.read_csv('customer_reviews.csv')
dataset = Dataset.from_pandas(df)

# Tokenize
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
def tokenize_function(examples):
    return tokenizer(examples["review"], padding="max_length",
        truncation=True, max_length=512)

tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Load model
model = AutoModelForSequenceClassification.from_pretrained("distilbert-
    base-uncased", num_labels=2)

# Training arguments
training_args = TrainingArguments(
```

```
    output_dir="./results",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    load_best_model_at_end=True,
    push_to_hub=False
)

# Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    eval_dataset=tokenized_dataset,
    tokenizer=tokenizer
)

# Train
trainer.train()

# Save model
model.save_pretrained("./fine_tuned_model")
tokenizer.save_pretrained("./fine_tuned_model")
```

10 Concentration: OpenAI API (GPT-4, GPT-3.5)

Listing 7: OpenAI API Integration

```
import openai
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

# Basic chat completion
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain quantum computing in
            simple terms."}
    ],
    temperature=0.7,
    max_tokens=500
)
print(response.choices[0].message.content)

# Streaming response
stream = client.chat.completions.create(
    model="gpt-4",
```

```
    messages=[{"role": "user", "content": "Write a poem about programming"}],
    stream=True
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="")

# Function calling (structured output)
functions = [
    {
        "name": "get_weather",
        "description": "Get current weather for a location",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string", "description": "City and country"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["location"]
        }
    }
]

response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "What's the weather in Tokyo?"}],
    functions=functions,
    function_call="auto"
)

# Image generation with DALL-E
response = client.images.generate(
    model="dall-e-3",
    prompt="A futuristic city with flying cars, cyberpunk style",
    size="1024x1024",
    quality="standard",
    n=1
)
image_url = response.data[0].url
print(image_url)

# Speech to text with Whisper
audio_file = open("speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file
)
print(transcription.text)

# Text to speech with TTS
response = client.audio.speech.create(
```

```
    model="tts-1",
    voice="alloy",
    input="Hello world! This is a test."
)
response.write_to_file("output.mp3")
```

11 RAG (Retrieval-Augmented Generation)

Listing 8: Document QA with RAG

```
from langchain.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# Load documents
loader = PyPDFLoader("company_handbook.pdf")
documents = loader.load()

# Split into chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", " ", ""]
)
chunks = text_splitter.split_documents(documents)

# Create embeddings and vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# Create retrieval chain
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-3.5-turbo", temperature=0),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(search_kwargs={"k": 3})
)

# Ask questions
question = "What is the company's policy on remote work?"
answer = qa_chain.run(question)
print(answer)

# Save vector store for later
vectorstore.save_local("faiss_index")

# Load existing vector store
loaded_vectorstore = FAISS.load_local("faiss_index", embeddings)
```

12 Deploying AI Models as APIs

Listing 9: FastAPI for Model Serving

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from transformers import pipeline
import torch

app = FastAPI()

# Load model at startup (once)
classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english",
    device=0 if torch.cuda.is_available() else -1
)

class TextRequest(BaseModel):
    text: str
    model_version: str = "default"

class SentimentResponse(BaseModel):
    label: str
    confidence: float

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

@app.post("/predict", response_model=SentimentResponse)
async def predict(request: TextRequest):
    if not request.text:
        raise HTTPException(status_code=400, detail="Text␣cannot␣be␣
            empty")

    result = classifier(request.text)[0]
    return SentimentResponse(
        label=result['label'],
        confidence=result['score']
    )

@app.post("/batch-predict")
async def batch_predict(request: list[TextRequest]):
    texts = [r.text for r in request]
    results = classifier(texts)
    return [{"label": r['label'], "confidence": r['score']} for r in
        results]

# Run with: uvicorn main:app --reload --port 8000
```

Listing 10: Dockerfile for AI Model

```dockerfile
FROM python:3.10-slim

WORKDIR /app
```

Python + TensorFlow + OpenAI API

```
4
5  # Install system dependencies
6  RUN apt-get update && apt-get install -y \
7      build-essential \
8      && rm -rf /var/lib/apt/lists/*
9
10 # Copy requirements and install
11 COPY requirements.txt .
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # Copy model files
15 COPY models/ ./models/
16 COPY app.py .
17
18 # Download model at build time (or mount volume)
19 RUN python -c "from␣transformers␣import␣pipeline;␣pipeline('sentiment-
       analysis')"
20
21 EXPOSE 8000
22
23 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

13 5-Year AI Learning Roadmap

Year	Focus
Year 1	Python programming, data structures, NumPy, Pandas, Matplotlib
Year 2	Statistics, probability, linear algebra basics, Scikit-learn, classical ML algorithms
Year 3	Deep learning fundamentals, TensorFlow/PyTorch, neural networks, CNNs, RNNs
Year 4	NLP, transformers, Hugging Face, OpenAI API, RAG, fine-tuning LLMs
Year 5	Production AI: MLOps, deploying models, AI agents, multimodal AI, research

14 AI Project Portfolio Checklist

1. **Level 1:** Data analysis notebook (Kaggle dataset)
2. **Level 2:** Classification model with Scikit-learn (deployed as API)
3. **Level 3:** Deep learning model (image classifier or text classifier)
4. **Level 4:** RAG application (chat with your documents)
5. **Level 5:** Full-stack AI app (React + Node.js + OpenAI API)